

Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

Alessandra Rossi

Docker compose - esercizio

Docker che utilizza **Flask**, **MySQL**, e **Apache**

Struttura del progetto:

```
|
|├─ app/
|   │├─ main.py           # Codice principale dell'app Flask
|   │├─ requirements.txt  # Dipendenze Python
|   │├─ Dockerfile        # Dockerfile per creare l'immagine Flask
|   │└─ templates/
|       └─ index.html     # Il file HTML con il form per inserire il nome
|
|├─ apache/
|   └─ apache.conf        # Configurazione Apache
|
|├─ docker-compose.yml    # Configurazione di Docker Compose
|└─ .env                  # Variabili d'ambiente (es. credenziali MySQL)
```

Docker compose - config apache

Nota che qui Apache è creato come reverse proxy: viene usato per “schermare” le richieste in entrata e in uscita da uno o più server. In questo caso, quindi, viene implementato come misura di sicurezza dei server

```
<VirtualHost *:80>
    ServerName localhost

    # Configura il reverse proxy verso il server Flask
    ProxyPass / http://flask_app:5000/
    ProxyPassReverse / http://flask_app:5000/

    # Abilita i moduli necessari
    LoadModule proxy_module modules/mod_proxy.so
    LoadModule proxy_http_module modules/mod_proxy_http.so

    ErrorLog /var/log/apache2/error.log
    CustomLog /var/log/apache2/access.log combined
</VirtualHost>
```


Requirements

Come requirements, cioè nel file `app/requirements.txt`, abbiamo bisogno dei seguenti:

Flask
Flask_SQLAlchemy
pymysql

Soluzione: Flask - app/main.py

```
from flask import Flask, jsonify, request, render_template, redirect, url_for
from flask_sqlalchemy import SQLAlchemy
import os

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = (
    f"mysql+pymysql:// {os.getenv('MYSQL_USER')}:{os.getenv('MYSQL_PASSWORD')}}"
    f"@{os.getenv('MYSQL_HOST')}/{os.getenv('MYSQL_DATABASE')}}"
)
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

# Modello per gli utenti
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50), nullable=False)

@app.route('/')
def index():
    # Recupera tutti gli utenti dal database
    users = User.query.all()
    return render_template('index.html', users=users)

@app.route('/submit', methods=['POST'])
def submit():
    # Recupera il nome dal form e salva nel database
    name = request.form.get('name')
    if name:
        user = User(name=name)
        db.session.add(user)
        db.session.commit()
    return redirect(url_for('index')) # Ricarica la pagina per vedere la lista aggiornata

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Soluzione: app/templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Inserisci Nome</title>
</head>
<body>
  <h1>Inserisci il tuo nome</h1>
  <form method="POST" action="/submit">
    <label for="name">Nome:</label>
    <input type="text" id="name" name="name" required>
    <button type="submit">Invia</button>
  </form>

  <h2>Lista degli utenti:</h2>
  <ul id="user-list">
    {% for user in users %}
      <li>{{ user.name }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```


Soluzione: app/Dockerfile

Usa un'immagine Python con supporto ARM
FROM python:3.10-slim

Imposta la directory di lavoro
WORKDIR /app

Copia i file del progetto
COPY . /app

Installa le dipendenze
RUN pip install -r requirements.txt

Espone la porta 5000
EXPOSE 5000

Comando di avvio dell'app
CMD ["python", "main.py"]



Questo è compatibile con
osx, per altre versioni/
architetture, modificare
l'immagine scelta

Soluzione: docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
  mysql:
```

```
    image: mysql:8
```

```
    platform: linux/amd64 # Forza Docker a usare l'architettura x86 (se necessario)
```

```
    environment:
```

```
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
```

```
      MYSQL_DATABASE: ${MYSQL_DATABASE}
```

```
      MYSQL_USER: ${MYSQL_USER}
```

```
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
```

```
    volumes:
```

```
      - mysql_data:/var/lib/mysql
```

```
    networks:
```

```
      - webnet
```

```
  flask_app:
```

```
    build:
```

```
      context: ./app
```

```
    platform: linux/amd64 # Forza Docker a usare l'architettura x86 (se necessario)
```

```
    environment:
```

```
      MYSQL_HOST: mysql
```

```
      MYSQL_DATABASE: ${MYSQL_DATABASE}
```

```
      MYSQL_USER: ${MYSQL_USER}
```

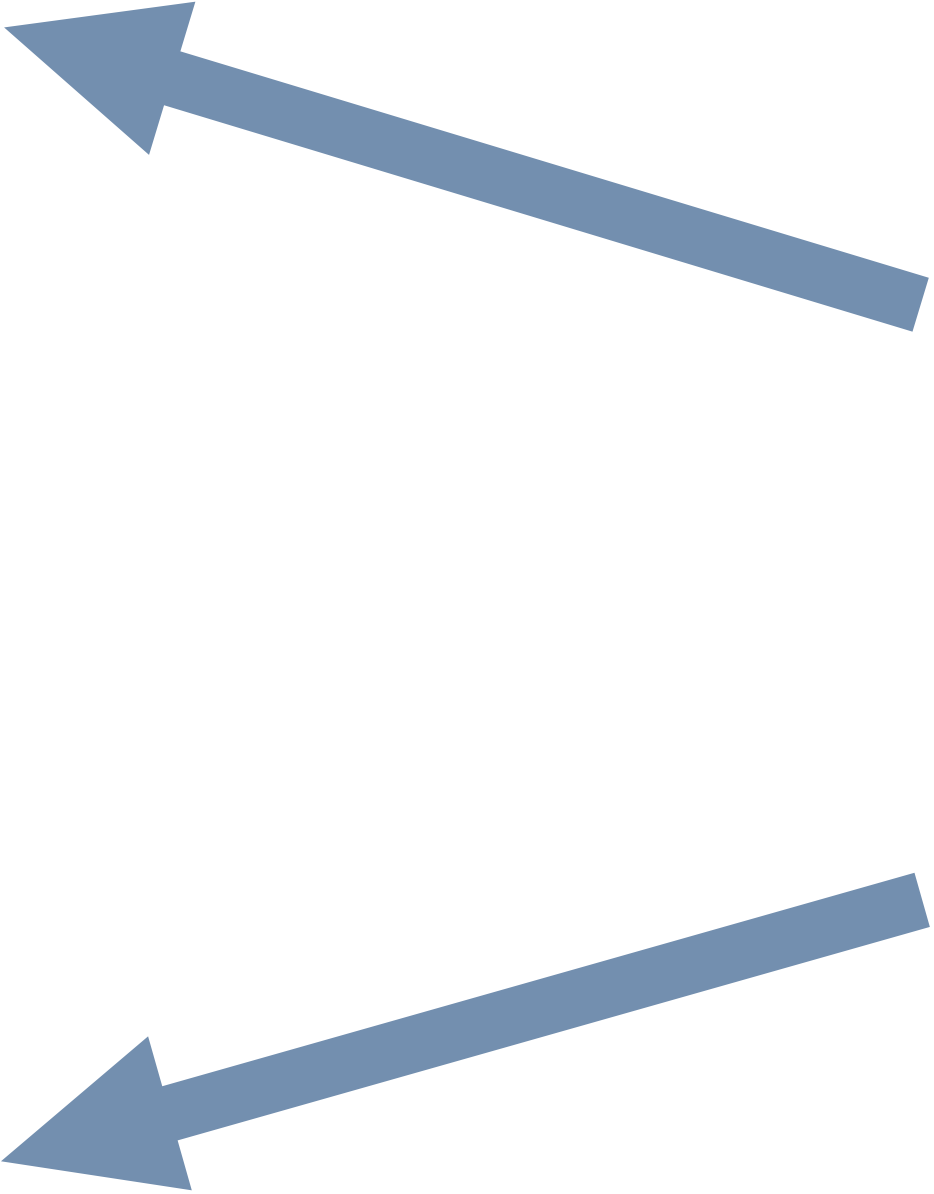
```
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
```

```
    depends_on:
```

```
      - mysql
```

```
    networks:
```

```
      - webnet
```



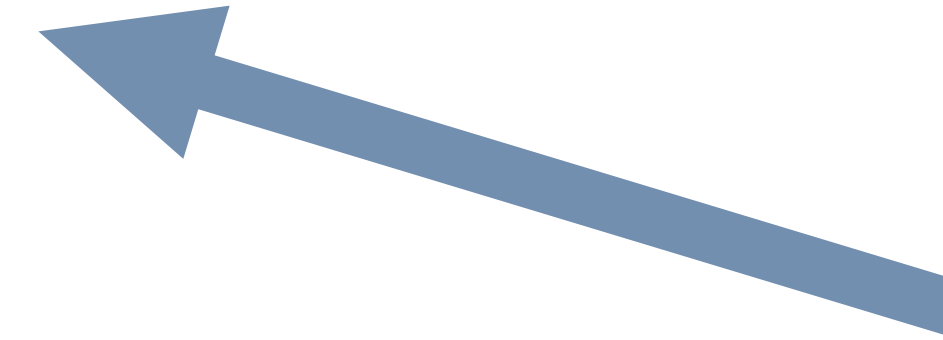
Questo è compatibile con
osx, per altre versioni/
architetture, si può
eliminare

Soluzione: docker-compose.yml

```
apache:
  image: httpd:2.4
  platform: linux/amd64 # Forza Docker a usare l'architettura x86 (se necessario)
  volumes:
    - ./apache/apache.conf:/usr/local/apache2/conf/httpd.conf
  ports:
    - "80:80"
  depends_on:
    - flask_app
  networks:
    - webnet
```

```
networks:
  webnet:
```

```
volumes:
  mysql_data:
```



Questo è compatibile con
osx, per altre versioni/
architetture, si può
eliminare

Soluzione: .env

MYSQL_ROOT_PASSWORD=rootpassword

MYSQL_DATABASE=mydatabase

MYSQL_USER=myuser

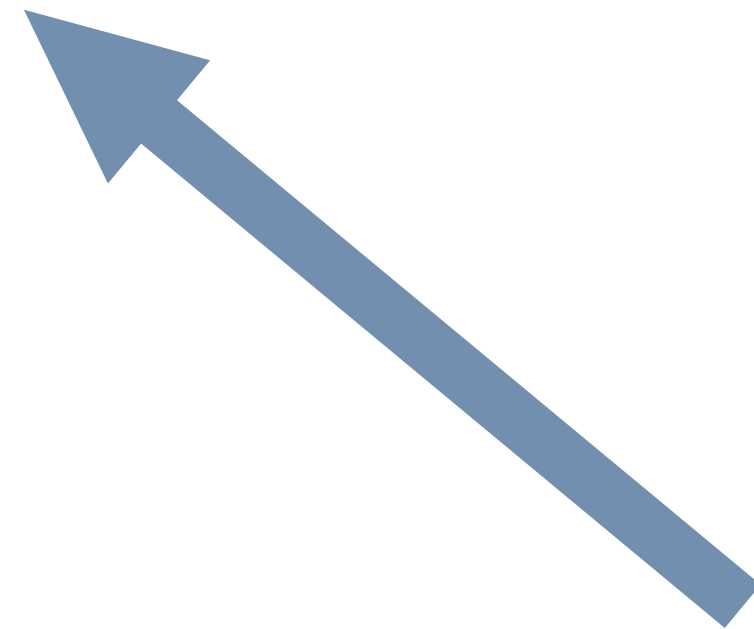
MYSQL_PASSWORD=mypassword

Soluzione: testing & running

- `docker-compose up --build`
- Vai nel browser e apri <http://localhost>
- `docker-compose down`
- `docker-compose logs -f`



Ferma il container



Visualizza i log

Esercizio Bash

- (a) Utilizzando awk si mostri la prima parola di tutte le linee di un file che non contengono una y.
- (b) Mostrare le linee di un file che contengono una delle due condizioni: 1) contengono "lso" oppure "esame" non considerando il sensitive case o l'ordine; contengono "BSD" o "ASD" considerando il sensitive case ma non l'ordine
- (c) Dato un file addr.txt, sostituire ogni terza parola con il numero "42"
- (d) Stampare le prime 5 righe di tutti file creati nella giornata di oggi.

Manuale per (d), possibili opzioni / soluzioni:

`-mtime n[smhdw]`

If no units are specified, this primary evaluates to true if the difference between the file last modification time and the time find was started, rounded up to the next full 24-hour period, is n 24-hour periods.

If units are specified, this primary evaluates to true if the difference between the file last modification time and the time find was started is exactly n units.

Please refer to the `-atime` primary description for information on supported time units.

`-newer file`

True if the current file has a more recent last modification time than file.

`find / -newer ttt -user wnj -print`

Print out a list of all the files owned by user "wnj" that are newer than the file ttt.

`find / \! \( -newer ttt -user wnj \) -print`

Print out a list of all the files which are not both newer than ttt and owned by "wnj".

`find / \( -newer ttt -or -user wnj \) -print`

Print out a list of all the files that are either owned by "wnj" or that are newer than ttt.

`find / -newerct '1 minute ago' -print`

Print out a list of all the files whose inode change time is more recent than the current time minus one minute.

Esercizio Bash

- (a) `awk '!/y/ {print $1}' nome_del_file`
- (b) `awk '$0 ~ /[lL][sS][oO]/ || $0 ~ /[eE][sS][aA][mM][eE]/ || $0 ~ /BSD/ || $0 ~ /ASD/ && $0 !~ /asd/ { print $0 }' testo.tx`
- (c) `awk '{for (i=1; i<=NF; i++) if (i % 3 == 0) $i = 42}1' addr.txt`
- (d) `find . -type f -newermt $(date +%F) -exec head -n 5 {} \;` **OPPURE** `find . -type f -mtime -1 -exec head -n 5 {} \;`



Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

